



MÓDULO 3 · T09

useEffect y Estados de Carga

⌚ *Loading · Data · Error*

Carlos Rodríguez Contreras · GitHub: `karrocon`

¿Qué aprenderás hoy?

En esta sesión aprenderás a conectar tus componentes React Native con datos reales mediante fetch asíncrono. Al terminar, sabrás modelar los tres estados de cualquier petición y construir una experiencia de usuario robusta frente a errores y tiempos de espera.

1

useEffect al montar

Usar `useEffect` para disparar el fetch cuando el componente se monta en pantalla.

2

3 estados del fetch

Modelar `loading`, `data` y `error` – nunca asumir que la petición siempre tiene éxito.

3

Indicadores de carga

Mostrar un `ActivityIndicator` mientras carga y un mensaje de error si falla.

4

Dependencias

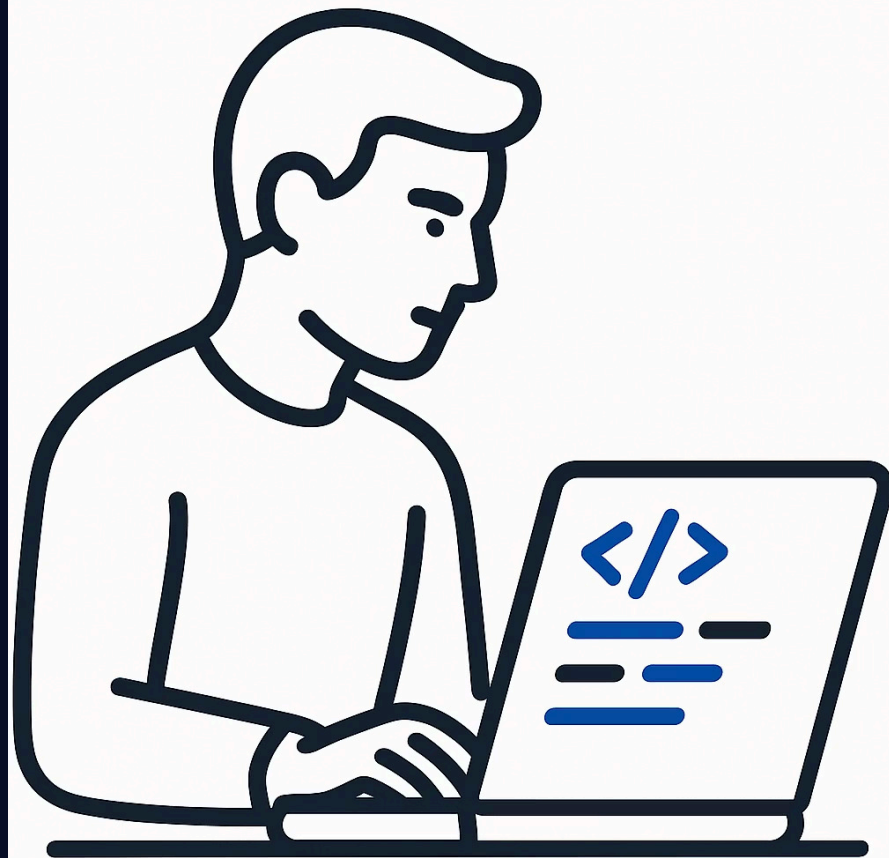
Gestionar el array de dependencias de `useEffect` para re-ejecutar el fetch cuando cambian los parámetros.

useEffect — ejecución al montar

El hook `useEffect` acepta una función que React ejecuta **después del primer render**. El segundo argumento es el array de dependencias: cuando está vacío, el efecto solo se ejecuta una vez – exactamente lo que necesitamos para lanzar el fetch inicial.

```
useEffect(() => {  
  // Se ejecuta después del primer render  
  fetchData();  
}, []); // ← [] = solo al montar
```

❗ El array `[]` vacío significa "ejecuta solo una vez". Si añades variables al array, el efecto se re-ejecuta cada vez que esas variables cambian.



Los 3 estados del fetch

¿Por qué tres estados?

Toda petición de red puede terminar de tres maneras distintas: en progreso, con éxito o con error. Modelar los tres desde el principio evita interfaces que se quedan en blanco o que muestran datos obsoletos.

Si solo manejas el caso de éxito, tus usuarios verán pantallas rotas cuando la conexión falle – ¡no esperes a producción para descubrirlo!

- ✓ Usar `finally` garantiza que `isLoading` siempre se pone a `false`, tanto en éxito como en error.

Declaración de los estados

```
const [pokemons, setPokemons] =  
  useState<Pokemon[]>([]);  
  
// ← empieza cargando  
const [isLoading, setIsLoading] =  
  useState(true);  
  
const [isError, setIsError] =  
  useState(false);
```

Siempre debes modelar los **tres** – nunca asumas éxito.

Fetch en useEffect

La función asíncrona se declara **dentro** del efecto y se invoca inmediatamente. El bloque `try / catch / finally` cubre los tres estados: activa el spinner al inicio, guarda los datos en caso de éxito, marca el error si falla, y desactiva el spinner siempre al final.

```
useEffect(() => {
  setIsLoading(true);
  setIsError(false);

  async function fetchData() {
    try {
      const data = await fetchPokemons(20);
      setPokemons(data);
    } catch {
      setIsError(true);
    } finally {
      setIsLoading(false);
    }
  }

  fetchData();
}, []);
```

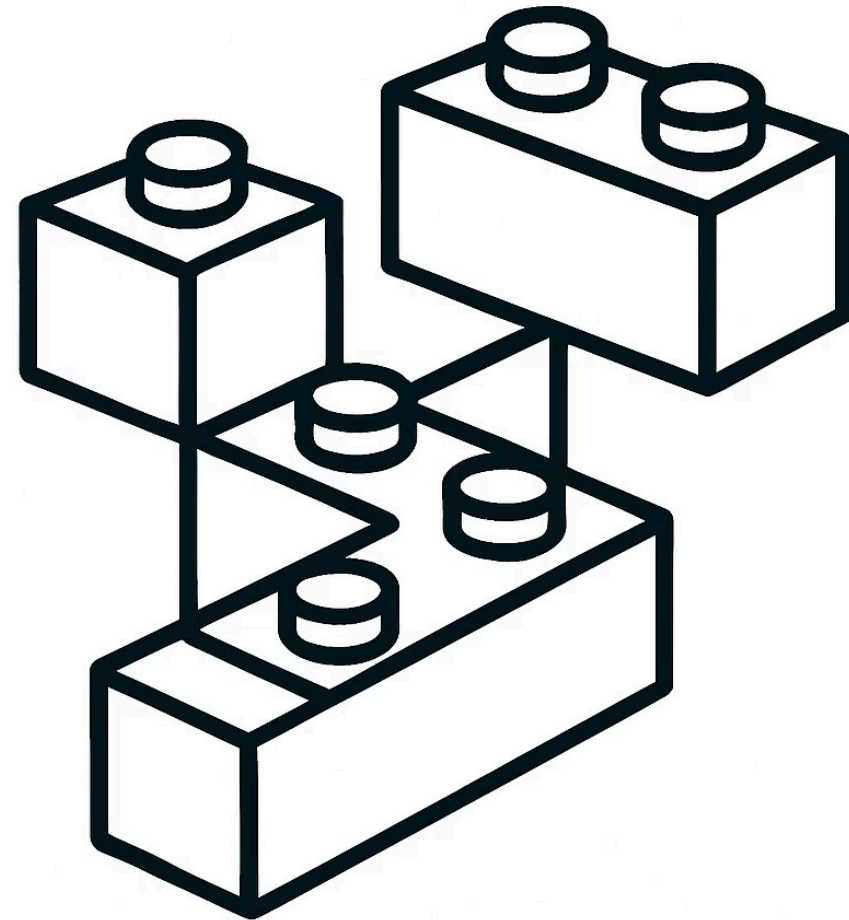
Renderizado condicional

Con los tres estados declarados, el componente devuelve una UI diferente según el momento. Los **early returns** hacen el código más legible: si hay carga o error, salimos antes de llegar al `FlatList` principal.

```
if (isLoading)
  return (
    <ActivityIndicator
      size="large"
      color="#EF5350"
    />
  );

if (isError)
  return (
    <Text style={styles.error}>
      Error al cargar
    </Text>
  );

return (
  <FlatList
    data={pokemons}
    renderItem={({ item }) => <PokemonListItem pokemon={item} />}
    keyExtractor={(item) => item.id.toString()}
  />
);
```



useEffect con dependencias

Cuando añades una variable al array de dependencias, React re-ejecuta el efecto cada vez que esa variable cambia. Esto es perfecto para paginación: al incrementar `page`, el fetch se lanza automáticamente sin necesidad de llamarlo manualmente.

```
const [page, setPage] = useState(1);

useEffect(() => {
  fetchPokemons(20, (page - 1) * 20)
    .then(setPokemons);
}, [page]); // ← re-carga cuando cambia `page`

// Cambiar page dispara la recarga:
<Button
  title="Siguiente"
  onPress={() => setPage(p => p + 1)}
/>
```

- ❗ Cada variable que usas dentro del efecto y que puede cambiar debe declararse en el array de dependencias – el linter de ESLint (`exhaustive-deps`) te avisará si olvidas alguna.

PokeApp — usePokemons.ts (estructura)

El hook personalizado encapsula toda la lógica de carga en un solo lugar. El componente que lo consuma solo necesita desestructurar el resultado – sin saber nada de `fetch`, estados internos ni efectos. Observa el cuarto estado `loadingPercentage` para la barra de progreso paginada.

```
export default function usePokemons({
  page,
  pageSize,
  fetchAllPages
}) {
  const [pokemons, setPokemons] = useState<Pokemon[]>([]);
  const [isLoading, setIsLoading] = useState(true);
  const [loadingPercentage, setLoadingPercentage] = useState(0);
  const [isError, setIsError] = useState(false);

  useEffect(() => {
    setIsLoading(true);
    setLoadingPercentage(0);
    fetchPokemons();
  }, [page, pageSize]);

  return { pokemons, isLoading, loadingPercentage, isError };
}
```

📌 **Buena práctica:** extrae la lógica de fetch a un custom hook (use...) para mantener tus componentes limpios y reutilizar la lógica fácilmente en otras pantallas.

Barra de progreso

Cálculo del porcentaje

```
// PokeApp: progreso de carga paginada
setLoadingPercentage(
  ((offset + pokemonsInPage.length) / total) * 100
);
```

Se actualiza en cada bloque recibido, dividiendo los Pokémon ya cargados entre el total devuelto por la API.

Renderizado de la barra

```
<View style={styles.bar}>
  <View
    style={[
      styles.fill,
      { width: `${loadingPercentage}%` }
    ]}
  />
</View>
```

La barra es un `View` anidado cuyo ancho en porcentaje refleja directamente el estado `loadingPercentage`.

❗ La barra de progreso solo tiene sentido cuando `fetchAllPages` está activo y la carga es incremental – en una sola página un `ActivityIndicator` simple suele ser suficiente.

Tour starter/usePokemonsSimple.ts

Este es el fichero que encontrarás en la carpeta `starter/`. Contiene la estructura básica del hook con los tres estados ya declarados y un `TODO` dentro del `useEffect` que deberás completar durante el ejercicio. Los estados de carga y error ya tienen su tipo inferido – solo falta implementar el `fetch`.

```
export default function usePokemonsSimple() : UseResult {  
  const [pokemons, setPokemons] =  
    useState<Pokemon[]>([]);  
  const [isLoading, setIsLoading] =  
    useState(true);  
  const [isError, setIsError] =  
    useState(false);  
  
  useEffect(() => {  
    // TODO: fetch aquí  
  }, []);  
  
  return { pokemons, isLoading, isError };  
}
```

⚠ No modifiques la firma del hook ni los nombres de los estados – el componente que lo consume ya espera exactamente estos tres valores.

Ejercicio T09

Pon en práctica todo lo aprendido. Abre el proyecto en tu editor y sigue los pasos en orden – cada uno construye sobre el anterior. El bonus es opcional pero muy recomendable para afianzar el patrón de dependencias en `useEffect`.

1 Abre el fichero de inicio

Navega a `starter/usePokemonsSimple.ts` en tu editor.

2 Implementa el fetch

Completa el `TODO` dentro de `useEffect` con la llamada asíncrona a `fetchPokemons`.

3 Gestiona los 3 estados

Usa `setIsLoading`, `setIsError` y `setPokemons` con `try / catch / finally`.

4 Renderizado condicional

Usa el hook en un componente y añade early returns para `isLoading` e `isError`.

5 Bonus: estado `page`

Añade un estado `page` y agrégalo al array de dependencias para recargar al cambiar de página.

Resumen de la sesión

Estos son los conceptos clave que has trabajado hoy. Dominarlos te permitirá conectar cualquier componente React Native con una API real, gestionando todos los estados posibles de una petición de forma robusta y predecible.

Concepto	Sintaxis clave	Para qué sirve
<code>useEffect(() => {}, [])</code>	Array vacío	Ejecuta solo al montar el componente
<code>useEffect(() => {}, [dep])</code>	Array con variable	Re-ejecuta al cambiar la dependencia
<code>isLoading</code>	<code>useState(true)</code>	Muestra spinner mientras carga
<code>isError</code>	<code>useState(false)</code>	Muestra mensaje de error si falla
<code>finally</code>	<code>setIsLoading(false)</code>	Siempre limpia el estado de carga
<code>ActivityIndicator</code>	<code>size="large"</code>	Spinner nativo multiplataforma

✅ ¡Enhorabuena! Ahora tienes las herramientas para conectar cualquier componente con una API real, modelando correctamente los tres estados del ciclo de vida de una petición. 🚀